Product Requirements Document

Vibe Code Security Scanner

Version: 1.0
Author: Jude Perera
Project: Vibe Code Security Scanner
Date: August 2026

1. Product Overview

1.1 Purpose

This document defines the product requirements for the Vibe Code Security Scanner — a web application that automatically detects security vulnerabilities introduced by AI-generated (vibe-coded) codebases. It is intended for security engineers, developers, and technical interviewers reviewing the project.

1.2 Problem Statement

AI code generation tools produce working applications quickly, but they consistently omit critical security controls. These omissions are not code bugs — they are system design failures that standard linters and code reviews do not catch. They include missing authentication on API routes, hardcoded secrets, SQL injection vulnerabilities, tokens stored in localStorage, and absent rate limiting on sensitive endpoints.

These failures are invisible until exploited. There is no lightweight, open-source tool focused specifically on detecting the security patterns that AI-generated code consistently misses.

1.3 Product Vision

A web-based security scanner that accepts any public GitHub repository URL, automatically analyses the codebase for 14 categories of security vulnerability, and presents findings in a severity-ranked dashboard with full scan history.

1.4 Target Users

- Security engineers reviewing AI-generated codebases before deployment
- Developers who use AI tools to build applications and want to verify security posture
- DevSecOps teams integrating security scanning into CI/CD pipelines
- Security engineering students building a practical portfolio project

---

2. Features

2.1 Core Features

F1 — Repository Scanning

- Accept a public GitHub repository URL via a web form
- Clone the repository to a temporary directory
- Scan all Python, HTML, and JavaScript files
- Delete the cloned repo after scanning — nothing persists on disk
- Return findings within 60 seconds for repos under 10,000 lines

F2 — Security Checks

- 14 security checks across three file categories
- Python AST-based checks for application and system design vulnerabilities
- HTML and JavaScript regex-based checks for frontend security issues
- Each finding includes: check name, severity, file path, line number, plain English detail

F3 — Severity Ranking

- Three severity levels: HIGH, MEDIUM, LOW
- Findings displayed in severity order — HIGH first

- Summary counts shown per severity level
- Colour-coded badges: red for HIGH, amber for MEDIUM, green for LOW

F4 — Web Dashboard

- Single-page web interface accessible via browser
- GitHub URL input form with Run Scan button
- Loading state shown while scan runs
- Results table with columns: Severity, Check Name, File Path, Line Number, Detail
- Scan history section showing all previous scans
- Click any historical scan to reload its findings

F5 — Scan History

- Every scan saved to SQLite database with timestamp
- History panel shows repo URL, scan date, total findings
- Findings retrievable per scan indefinitely
- Vulnerability rate calculated as findings per scanned file

F6 — CI/CD Integration

- GitHub Actions pipeline with 5 automated jobs
- Bandit SAST, Semgrep OWASP rules, dependency audit, Gitleaks secret scan
- Unit tests run on every push — pipeline fails if tests fail
- All jobs run in parallel for speed

2.2 Security Checks — Full List

Check Name	Category	Severity	File Type
Hardcoded Secret	Application Security	HIGH	Python
SQL Injection Risk	Application Security	HIGH	Python
Missing Authentication	Application Security	HIGH	Python
Debug Mode Enabled	Application Security	MEDIUM	Python
Missing Input Validation	Application Security	MEDIUM	Python
Sensitive Data in Response	Application Security	HIGH	Python
Stack Trace Exposure	Application Security	MEDIUM	Python
Environment Variable in Source	Application Security	HIGH	Python
Missing Rate Limiting	System Design	MEDIUM/HIGH	Python
Missing Security Headers	System Design	MEDIUM	Python
Exposed Admin Endpoint	System Design	HIGH	Python
IDOR Risk	System Design	HIGH	Python
Missing File Upload Validation	System Design	HIGH	Python
Row Level Security Missing	System Design	HIGH	Python
Missing HTTPS Enforcement	System Design	HIGH	Python
Missing 2FA	System Design	MEDIUM	Python
Missing Server Side Validation	System Design	MEDIUM	Python
Outdated Dependencies	Dependencies	HIGH/MEDIUM	requirements.txt
API Key Exposed on	Frontend	HIGH	HTML/JS

Check Name	Category	Severity	File Type
Frontend			
Insecure Token Storage	Frontend	HIGH	HTML/JS

---

3. Non-Functional Requirements

3.1 Performance

- Scan completes within 60 seconds for repositories under 10,000 lines of code
- Dashboard loads in under 2 seconds
- Scan history loads in under 1 second

3.2 Security

- Cloned repositories deleted immediately after scanning
- No user authentication required — public tool
- No sensitive data stored — only findings and repo URLs
- CI/CD pipeline scans the scanner's own code on every push

3.3 Reliability

- Graceful error handling if git clone fails
- Clear error messages returned to the user
- Temporary directories cleaned up on success and failure

3.4 Maintainability

- Checks separated into python_checks.py and frontend_checks.py
- Each check is an independent function — easy to add or remove
- Unit tests cover all core check functions
- 6 tests passing, all checks independently testable

4. Out of Scope

- User authentication or accounts
- Private repository scanning
- Dynamic analysis or runtime testing
- Automatic vulnerability remediation
- Support for languages other than Python, HTML, JavaScript
- Mobile application